

110.4. Devant, derrière : le tuyau, DERRIERE

`sprite(0, x, y, HEROS, DERRIERE)` : le décor passe devant le lutin. Le héros entre dans un tuyau et ressort de l'autre côté.

1. Ce qu'il faut comprendre

Un héros qui entre dans un tuyau doit disparaître DEDANS, et pas glisser par-dessus. Il faut que le tuyau passe devant lui.

Ce qui est nouveau ici : `DERRIERE`, le 5e argument de `sprite()`. On l'a déjà rencontré avec `MIROIR_X` (le miroir) : c'est la même place. `sprite(0, x, 72, HEROS, DERRIERE)` range ce lutin **derrière le décor**.

Mais pas derrière tout le décor ! Le décor ne cache le lutin que là où il est **dessiné**, avec les indices 1, 2 ou 3. Là où le décor est **vide** (l'indice 0, le fond clair de l'écran), le lutin reste visible. C'est pour ça qu'on voit le héros avant et après le tuyau : là, il n'y a que du vide.

Le tuyau est tout plein : ses tuiles n'ont aucun 0. Le héros y disparaît donc entièrement. Si le tuyau avait des trous (des 0), on verrait le héros à travers.

DERRIERE passe aussi derrière le panneau, là où le panneau est dessiné : pour la console, le panneau est un second décor.

À toi : ôte `DERRIERE` (garde `sprite(0, x, 72, HEROS);`) : le héros passe par-dessus le tuyau, comme dans la leçon d'avant.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **506 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	103 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide

```

CopierTuiles      76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
Tuiles            128 o ← CopierTuiles – 6 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert  10 o ← InstallerTransfert – la petite routine qui copie les lutins
"LE TUYAU"        8 o ← texte() – un texte gravé

```

Lettres gravées : AELTUY

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```

sprite() × 1    24 o sur place, dans main()
poser() × 1     13 o sur place, dans main() + AdresseCase (10 o)
texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1      6 o sur place, dans main() + AttendreImage (28 o)
images() × 1     2 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros avance tout seul et ENTRE dans un tuyau couché :
// il disparaît dedans, et ressort de l'autre côté.
//
// Ce qui est nouveau : DERRIERE, le 5e argument de sprite() (la place de MIROIR_X).
// sprite(0, x, y, HEROS);          le héros DEVANT le décor (comme d'habitude)
// sprite(0, x, y, HEROS, DERRIERE); le héros DERRIÈRE le décor
//
// Attention : le décor ne cache le lutin que là où il est DESSINÉ (1, 2 ou 3).
// Là où le décor est vide (0), on voit toujours le lutin.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
#include <texte>    // écrit un texte à l'écran
#include <sprite>   // place un lutin de 8 × 8 au pixel près

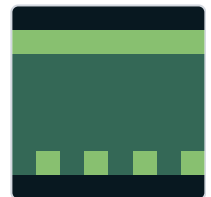
```

2. UN DESSIN — TUYAU

```

Tuile TUYAU = {                                // Un morceau de tuyau couché : AUCUN 0, il est tout plein.
    "33333333",                                // le bord du haut, sombre
    "11111111",                                // un reflet clair
    "22222222",
    "22222222",                                // le corps, gris
    "22222222",
    "22222222",
    "21212121",                                // l'ombre du bas
    "33333333",                                // le bord du bas
};

```



à l'écran

3. UN DESSIN — HEROS

```

Tuile HEROS = {                                // Le héros : les points (.) sont transparents.
    "..###..",
    ".#-##-#",
    "#####",
    "#.###.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};

```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "LE TUYAU");

    // Le tuyau : colonnes 8 à 12, ligne 9 (les pixels 64 à 103, en largeur).
    for (uint8_t c = 8; c < 13; c++) {
        poser(c, 9, TUYAU);
    }

    uint8_t x = 0;                          // La place du héros, en pixels.

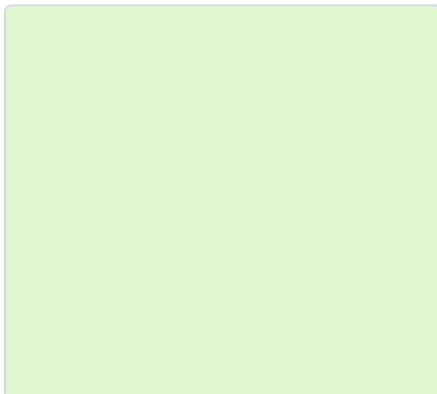
    while (true) {                          // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++;         // Un pixel toutes les 2 images.
        if (x > 152) x = 0;                 // Sorti à droite : il repart de la gauche.

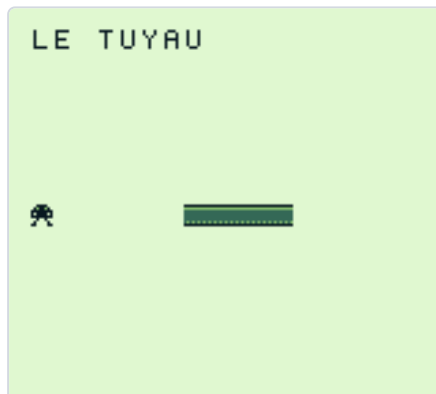
        // Ligne de pixels 72 = ligne 9 × 8 : pile à la hauteur du tuyau.
        sprite(0, x, 72, HEROS, DERRIERE); // DERRIERE : le tuyau passe devant lui.
    }
}
```

3. Ce qui se passe, étape par étape

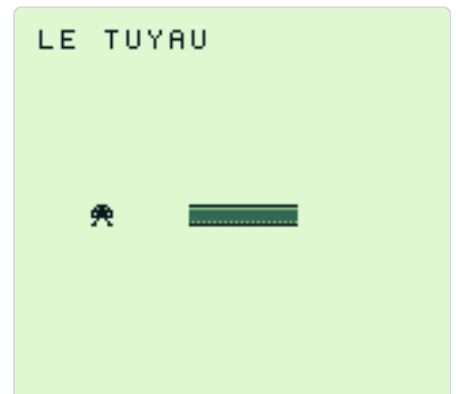
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



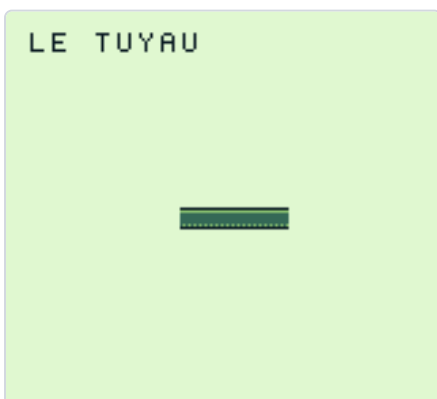
la toute première image — la console vient de démarrer — image 1



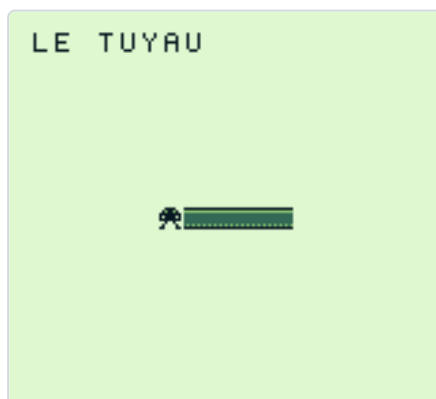
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un tuyau couché au milieu de l'écran. Le héros arrive de la gauche, disparaît dans le tuyau, et ressort à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ avant le tuyau, sur le vide, on voit le héros (nuance 3)

✓ dans le tuyau, il a disparu : on voit le tuyau (2)

✓ il ressort de l'autre côté

✓ le lutin porte bien DERRIERE

Ensuite : **110.5. Devant, derrière : deux lutins qui se croisent** — chapitre 17 : Le mouvement : les lutins au pixel près. ·
gameboy3 — écrire une cartouche Game Boy en C++